

Connector Layer — apm_connectors/tools/

The Python constructs this layer adds on top of the FastAPI/Action Graph docs, a from-zero tutorial on why a common connector interface exists, a complete deep dive into this repo's connectors, a per-tool reference covering all five (Gmail, Calendar, Excel, Salesforce, Jira) and how they interact, and the design patterns they use — written as a standalone reference, including for interview prep.

Part 0 — Python syntax this layer adds

Continues the FastAPI-layer doc's Part 0 and the Action Graph doc's Part 0 — read those first if imports, type hints, classes, decorators, closures, TypedDict, or @dataclass are still new. The connector layer adds six more constructs.

0.1 Abstract base classes — a contract subclasses must fulfill

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self) -> float:
        ...

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self) -> float:
        return 3.14159 * self.r ** 2

Shape()          # TypeError -- can't instantiate an ABC directly
Circle(2).area() # 12.566 -- fine, area() was implemented
```

A class inheriting from ABC can declare methods as @abstractmethod — no body, just a signature (and usually a docstring). Python then refuses to let you create an instance of any subclass that hasn't actually implemented every abstract method — you get a `TypeError` at the moment of instantiation, not a confusing failure later when the missing method finally gets called. This repo's `BaseTool(ABC)` uses exactly this to guarantee every connector implements `health_check` (§2.3) — forgetting to would be caught immediately, the first time anyone tries to construct that connector.

0.2 Protocol — an interface defined by shape, not by inheritance

```
from typing import Protocol

class Honks(Protocol):
    def honk(self) -> str: ...

class Car:
    # note: does NOT inherit from Honks at all
    def honk(self) -> str:
        return "beep"

def make_it_honk(x: Honks) -> str:
    # accepts ANYTHING with a honk() method
    return x.honk()

make_it_honk(Car()) # "beep" -- Car satisfies Honks just by having the right method
```

This is *structural* typing ("if it walks like a duck and honks like a duck...") as opposed to the *nominal* typing ABC uses (§0.1), where a class must explicitly say `class Car(Honks):` to count. A `Protocol` class is never instantiated and never inherited from — it's purely a shape description a type checker compares other classes

against. The ... (Python's Ellipsis literal) is a real, valid expression often used exactly this way — as a placeholder body meaning "no implementation here, this is just a signature." `gmail_tool.py`'s `GmailClient(Protocol)` (§2.4) is why both the real Gmail API wrapper and a test's fake client work as a `GmailTool`'s client argument without either one inheriting from anything — they just both happen to have matching methods.

0.3 Enum — a fixed, named set of values

```
from enum import Enum

class Capability(str, Enum):
    READ = "read"
    WRITE = "write"
    ACTION = "action"

Capability.READ          # Capability.READ
Capability.READ == "read" # True -- because it also inherits from str
```

An Enum restricts a value to one of a fixed, named set — `Capability.READ` is a real, distinct object, not just the string "read", which catches typos a plain string constant wouldn't (`Capability.RAED` is an `AttributeError` at the point of the typo; a misspelled string just silently fails to match later). Inheriting from `str` as *well as* `Enum` — `class Capability(str, Enum):` — is a common combination meaning "restricted to these values, but still comparable to and serializable as plain strings," useful anywhere the value needs to cross a boundary (JSON, a log line) that only understands plain strings.

0.4 A decorator that takes its own arguments

The FastAPI doc's §0.5 covered a plain `@decorator`. `@with_retry()` — note the parentheses — is one level more: a function that, when *called*, *returns* a decorator. Three layers, each with one job:

```
def with_retry(attempts=3):      # 1. called with config, e.g. with_retry(attempts=5)
    def decorator(func):        # 2. receives the function being decorated
        def wrapper(*args, **kwargs): # 3. runs on each actual call
            for i in range(attempts):
                try:
                    return func(*args, **kwargs)
                except OSError:
                    if i == attempts - 1:
                        raise
            return wrapper
        return decorator

# with_retry(attempts=5) runs immediately, returns `decorator`, which
# then wraps call_api exactly like a plain @decorator would
@with_retry(attempts=5)
def call_api():
    ...
```

`*args, **kwargs` in `wrapper`'s own signature is the receiving side of the unpacking the Action Graph doc covered (§0.5 there) — it means "accept any positional and keyword arguments at all," so this one wrapper can retry a call to *any* function, whatever arguments it happens to take. This is exactly this repo's `with_retry()` (§2.5), just simplified — the real one also uses `functools.wraps(func)` right above `wrapper`'s definition, which copies `func`'s original name/docstring onto `wrapper` — without it, every decorated method would misleadingly show up as a function literally named `wrapper` in stack traces and debuggers.

0.5 @staticmethod — a method that doesn't need self

```
class Converter:
    @staticmethod
    def celsius_to_fahrenheit(c: float) -> float:
        return c * 9 / 5 + 32    # doesn't touch `self` at all

Converter.celsius_to_fahrenheit(100)    # 212.0 -- no instance required
```

A regular method's first parameter is always `self` (FastAPI doc §0.4). `@staticmethod` removes that — the method doesn't receive an instance at all, because it doesn't need one; it's grouped inside the class purely for organization (it's conceptually "about" that class) rather than because it operates on a particular instance's data. `gmail_tool.py`'s `_to_summary` (§2.4) is exactly this shape — it transforms a raw dict into an `EmailSummary` without ever touching `self`.

0.6 A few smaller pieces you'll see in this layer

`frozenset({...})` — An immutable set — like a frozen tuple is to a list. `capabilities: frozenset[Capability]` means the set of what a connector can do is fixed at construction and can't be mutated afterward.

`f"{subject!r}"` — The `!r` inside an f-string (FastAPI doc §0.6) formats the value with `repr()` instead of `str()` -- for a string, that means it prints with quotes around it, making it obvious in a log line where the value starts and ends.

A dict comprehension over nested data — `{h["name"].lower(): h["value"] for h in raw.get("payload", {}).get("headers", [])}` -- same `[expr for item in iterable]` shape as a list comprehension (Action Graph doc §0.7 in the FastAPI doc's numbering), just building a dict instead of a list.

`TypeVar / Callable[... , T]` — `T = TypeVar("T")` declares a placeholder type used to say "this decorator preserves whatever type the wrapped function returns" -- `Callable[... , T]` means "a callable taking any arguments, returning T."

Part 1 — Connector-interface design, from zero

1.1 The problem: five external systems, one caller

Gmail, Google Calendar, Excel, Salesforce, and Jira each have their own SDK, their own authentication scheme, their own data shapes. Without a shared interface, the Action Graph's `execute_node` (see that doc, §2.3) would need to know, in detail, how to call each one — an `if/elif` chain checking which connector it's dealing with, one branch per system, growing every time a new connector is added. A **common interface** — a promise that every connector exposes the same shaped methods (§1.2) — lets calling code stay completely generic: `execute_node`'s `getattr(tool, proposed["method"])` (Action Graph doc §0.4) works identically whether `tool` is a `GmailTool` or a `JiraTool`, because both promise the same shape.

1.2 Interfaces as contracts, and two ways to express one in Python

An **interface**, in the general software-design sense, is a promise: "anything claiming to be an X will have these methods, with these signatures." Calling code writes against the interface, never caring which concrete implementation it was actually handed. This repo expresses that promise *two different ways*, deliberately, for two different relationships:

	ABC (nominal)	Protocol (structural)
How a class opts in	Explicit: <code>class GmailTool(BaseTool):</code>	Implicit: just have matching methods

Used for	BaseTool -- the connector's own public shape	GmailClient -- the HTTP client a connector wraps
Why this one here	Connectors are a small, closed set this repo owns and controls the shape of	A test fake and the real API client shouldn't have to share a base class just to both qualify

This contrast is worth sitting with (it reappears explicitly in §4): ABC is the right tool when you own every implementation and want Python to actively enforce completeness at construction time (§0.1). Protocol is the right tool when you don't want to force unrelated things (a real API client, a test fake) into a shared inheritance hierarchy just to prove they're interchangeable.

1.3 dry_run, as a general idea

A **dry run** is a call that goes through all the same logic — validation, logging, building the request — as a real call, but stops short of actually sending/creating/writing anything, and reports back what *would* have happened. It's a common pattern anywhere an action is expensive, dangerous, or irreversible enough that you want a way to preview it safely. This repo's twist: `dry_run` isn't an optional debugging flag here — it's the literal boolean that separates "describe this write" from "actually perform this write," and the entire human-approval mechanism (Action Graph doc) exists specifically to control when that flag is allowed to be `False`.

1.4 Idempotency, retries, and why writes can't be retried blindly

An operation is **idempotent** if doing it multiple times has the same effect as doing it once — GET-ing the same resource five times leaves the world in the same state as fetching it once. A network call can fail *after* the server did the work but *before* the response reaches the caller — from the caller's point of view, indistinguishable from the request never having arrived at all. Blindly retrying an idempotent read after that kind of failure is free and safe: worst case, you fetch the same data twice. Blindly retrying a *non*-idempotent write — sending an email, creating a record — risks doing it *twice*: once the server actually processed before the connection dropped, once more on the retry. This exact reasoning is why this repo's retry decorator (§2.5) is deliberately applied only to read methods, and the module docstring says so explicitly.

Part 2 — This repo's connector layer, section by section

File	Role
tools/base.py	Capability, ActionResult, BaseTool -- the shared shape every connector implements (88 lines).
tools/_retry.py	<code>with_retry</code> -- the read-only retry-with-backoff decorator factory (65 lines).
tools/gmail_tool.py	One concrete connector, used throughout this doc as the worked example (226 lines); the other four (calendar, excel_file, salesforce, jira) follow the same shape.

2.1 Capability — what a connector is allowed to be asked to do

```
class Capability(str, Enum):
    READ = "read"
    WRITE = "write"
    ACTION = "action"

# in GmailTool:
capabilities = frozenset({Capability.READ, Capability.ACTION})
```

Every connector declares its capabilities as a class attribute — Gmail can READ (search/read messages) and take an ACTION (send), but has no bare WRITE capability (that's used by connectors like Excel/Salesforce that

overwrite existing data rather than triggering a one-shot action). `can(capability)` (§2.3) is the one place this gets checked — mostly useful for a UI or caller wanting to know upfront what's possible without trying and catching a failure.

2.2 ActionResult — the uniform write/action return shape

```
@dataclass(frozen=True)
class ActionResult:
    """What a write/action call returns, whether it actually ran or
    was a dry run -- callers treat both the same shape, checking
    `executed` to tell them apart."""
    executed: bool
    description: str
    details: dict[str, Any]
```

Every write/action method across every connector — `send_email`, `create_event`, `write_range`, `create_record` — returns exactly this shape. The Action Graph's `execute_node` (that doc, §2.3) reads `action_result.executed` generically, with zero knowledge of which specific connector produced it. This is the connector-layer equivalent of `RunOutcome`'s "discriminated by convention" design from the Action Graph doc — one uniform result shape instead of one bespoke type per connector.

2.3 BaseTool — the shared contract

```
class BaseTool(ABC):
    name: str
    capabilities: frozenset[Capability]

    def __init__(self, state: StateStore) -> None:
        self._state = state

    def can(self, capability: Capability) -> bool:
        return capability in self.capabilities

    def _log(self, process_id, event_type, summary, details=None) -> None:
        self._state.log_event(process_id=process_id, tool=self.name,
                               event_type=event_type, summary=summary, details=details or {})

    @abstractmethod
    def health_check(self) -> bool:
        """... Must never raise -- return False on failure ..."""

    def require_dry_run_guard(self, dry_run: bool, process_id: str, summary: str) -> None:
        event_type = "action_executed" if not dry_run else "action_proposed"
        self._log(process_id, event_type, summary, {"dry_run": dry_run})
```

Four things every subclass inherits for free: a constructor that stashes the shared `StateStore`; `_log`, the single audit-trail write path every public method is expected to funnel through; `require_dry_run_guard`, called at the top of every write/action method (§1.3); and the `@abstractmethod` `health_check` every connector must implement (§0.1) — deliberately required to *never raise*, so a UI can poll every connector's status in a loop without one bad credential crashing the whole check.

2.4 GmailTool — the worked example

The Protocol-typed client (§0.2) — this is the entire surface `GmailTool` needs:

```
class GmailClient(Protocol):
    def list_message_ids(self, query: str, max_results: int) -> list[str]: ...
    def get_message(self, message_id: str) -> dict[str, Any]: ...
    def send_message(self, to: str, subject: str, body: str) -> dict[str, Any]: ...
```

A read method — runs immediately, logs a plain "read" event:

```
def search_emails(self, process_id: str, query: str, max_results: int = 10) ->
    list[EmailSummary]:
    message_ids = self._client.list_message_ids(query=query, max_results=max_results)
    summaries = [self._to_summary(self._client.get_message(mid)) for mid in message_ids]
    self._log(process_id, "read", f"Searched Gmail for '{query}', found {len(summaries)}
        message(s)",
        {"query": query, "count": len(summaries)})
    return summaries
```

The write method — three independent layers of defense, in order:

```
def send_email(self, process_id, to, subject, body, dry_run: bool = True) -> ActionResult:
    # 1. refuse fake domains outright
    if _is_reserved_placeholder_address(to):
        summary = f"Refused to send to {to}: reserved for documentation/examples"
        self._log(process_id, "action_failed", summary, {"to": to, "subject": subject})
        return ActionResult(executed=False, description=summary, details={...})

    summary = f"Send email to {to}: {subject!r}"
    # 2. log regardless of outcome
    self.require_dry_run_guard(dry_run, process_id, summary)

    # 3. dry_run is the actual gate
    if dry_run:
        return ActionResult(executed=False, description=summary, details={...})

    sent = self._client.send_message(to=to, subject=subject, body=body)
    return ActionResult(executed=True, description=summary, details={...})
```

`dry_run: bool = True` is a fail-safe default — call `send_email(...)` without explicitly passing `dry_run=False`, and nothing gets sent, no matter what. The one place that default gets overridden is the Action Graph's `execute_node`, hardcoding `dry_run=False` (that doc, §2.3) — and only after `interrupt()` has returned an approval.

2.5 The placeholder-domain guard — a concrete defense-in-depth example

"RFC 2606 reserves the .test/.example/.invalid/.localhost TLDs ... A safety net against the reasoner fabricating a plausible-looking placeholder recipient (observed in practice: it proposed 'customer@example.com' when no real customer address was present anywhere in the fetched data) -- this check runs regardless of what the reasoner outputs or whether a human approved it, since a human reviewing a payload has no reason to recognize 'example.com' as fake rather than a real domain they don't happen to know."

Worth reading closely because it's a real bug this guard was written in direct response to, not a hypothetical: an LLM-based reasoning layer, lacking a real recipient address in its input data, confidently invented a plausible-looking one. A human approver reviewing that payload has no way to distinguish a fabricated address from a real one they simply don't recognize — so the check can't live at the human-approval step; it has to be a hard refusal inside the connector itself, before `dry_run` is even considered. This is *defense in depth* as a concrete, motivated design choice: three independent layers (this domain check, `require_dry_run_guard`'s audit logging, and the Action Graph's `interrupt()` gate one level up) each catch a different failure mode, none of them redundant with the others.

2.6 `_retry.py` — read-only, on purpose

```

TRANSIENT_EXCEPTIONS = (OSError,) # covers ConnectionError, TimeoutError, etc.

def with_retry(attempts: int = 3, base_delay_seconds: float = 0.5):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, attempts + 1):
                try:
                    return func(*args, **kwargs)
                except TRANSIENT_EXCEPTIONS as exc:
                    last_error = exc
                    if attempt < attempts:
                        time.sleep(base_delay_seconds * attempt) # linear backoff
            raise last_error
        return wrapper
    return decorator

# usage, in GoogleApiGmailClient:
@with_retry()
def list_message_ids(self, query: str, max_results: int) -> list[str]:
    ...

# NOT decorated -- see the module docstring:
def send_message(self, to: str, subject: str, body: str) -> dict[str, Any]:
    ...

```

`TRANSIENT_EXCEPTIONS = (OSError,)` is deliberately narrow — it catches connection-level failures (the request never got a real response at all), but not HTTP error status codes like 429 or a 5xx, which arrive as a different, client-library-specific exception type carrying response details a caller may actually want to inspect rather than silently retry past. `list_message_ids` and `get_message` — both reads — get the decorator; `send_message` — a write — deliberately does not, straight out of the idempotency reasoning in §1.4.

2.7 Lazy imports and factory functions

```

class GoogleApiGmailClient:
    def __init__(self, credentials: Any) -> None:
        from googleapiclient.discovery import build # imported HERE, not at module top
        self._service = build("gmail", "v1", credentials=credentials)

    def build_gmail_tool(state: StateStore, settings: Settings) -> GmailTool:
        from apm_connectors.tools.google_auth import load_credentials
        credentials = load_credentials(settings, scopes=[...])
        return GmailTool(state, GoogleApiGmailClient(credentials))

```

Same local-import pattern the FastAPI doc's §0.10 covered for the Postgres driver, here applied to `googleapiclient`: importing `gmail_tool` — including for unit tests, which construct `GmailTool` directly with a fake client (§0.2) — never requires `googleapiclient` to be installed at all; only actually building a `GoogleApiGmailClient` does. `build_gmail_tool` is the same factory-function shape as `create_app()` and `build_action_graph(...)` from the other two docs — take real dependencies (settings, a state store), run whatever setup is needed (the OAuth flow), return a fully-built object — used by the running app, deliberately bypassed by tests in favor of constructing `GmailTool` directly.

Part 3 — The other four connectors, and how the whole layer interacts

Gmail (§2.4-2.6) was the worked example because it's the smallest complete one. The other four — Calendar, Excel, Salesforce, Jira — follow the exact same `BaseTool` shape (§2.3): a Protocol-typed client, a dataclass for

normalized results, read methods that run immediately, write/action methods gated by `dry_run + require_dry_run_guard`. What's actually worth knowing about each is where it differs — its auth mechanism and its specific quirks — plus how all five get wired together into one running server.

3.1 Google Calendar — the same shape as Gmail, one write method

```
class CalendarClient(Protocol):
    def list_events(self, time_min, time_max, query, max_results) -> list[dict[str, Any]]: ...
    def get_event(self, event_id: str) -> dict[str, Any]: ...
    def insert_event(self, payload: dict[str, Any]) -> dict[str, Any]: ...

class CalendarTool(BaseTool):
    name = "google_calendar"
    capabilities = frozenset({Capability.READ, Capability.ACTION})
```

`search_events/read_event` are reads; `create_event` is the one write, single (non-recurring) events only, RFC3339 datetimes. ACTION rather than plain WRITE here — same as Gmail — because creating an event, like sending an email, triggers a one-shot side effect rather than overwriting something that already exists (contrast with Excel/Salesforce/Jira below, §3.5).

3.2 Excel (local file or Google Drive) — one tool, two interchangeable sources

```
class WorkbookSource(Protocol):
    def describe(self) -> str: ...
    def read_bytes(self) -> bytes: ...
    def write_bytes(self, data: bytes) -> None: ...

class ExcelFileTool(BaseTool):
    name = "excel_file"
    capabilities = frozenset({Capability.READ, Capability.WRITE})
    def __init__(self, state: StateStore, source: WorkbookSource) -> None:
        super().__init__(state)
        self._source = source      # LocalWorkbookSource or GoogleDriveWorkbookSource
```

`ExcelFileTool` never knows or cares whether its bytes come from a local path or a Drive file — it only ever calls `read_bytes/write_bytes` on whichever `WorkbookSource` it was built with, then parses/edits those bytes with `openpyxl` (there's no cell-range API to call for either source). Two real quirks worth knowing: `read_range` loads with `data_only=True`, so a formula cell that's never been opened in real Excel/Sheets has no cached value and reads back as `None`; and the Drive source requests the full drive scope rather than the narrower `drive.file` scope, because `drive.file` only grants access to files the app itself created — confirmed live, not just reasoned about, by a 403 `insufficientPermissions` on write.

3.3 Salesforce — server-to-server OAuth, no interactive consent

```
class SalesforceClient(Protocol):
    def query(self, soql: str) -> list[dict[str, Any]]: ...
    def get_record(self, object_name: str, record_id: str) -> dict[str, Any]: ...
    def create_record(self, object_name: str, fields: dict[str, Any]) -> dict[str, Any]: ...
    def update_record(self, object_name, record_id, fields) -> dict[str, Any]: ...

class SalesforceTool(BaseTool):
    name = "salesforce"
    capabilities = frozenset({Capability.READ, Capability.WRITE})
```

`query_records` runs a SOQL query; `create_record/update_record` are the writes. Unlike Gmail/Calendar's user-consent OAuth (§2.7), Salesforce authenticates with OAuth 2.0's **Client Credentials Flow** against a Connected App — no browser, no redirect URI, no human logging in; it's the app itself authenticating as a pre-configured "Run As" user. `build_configured_salesforce_tool` acquires that

access token *eagerly*, at server startup, rather than lazily on first call — a misconfigured Connected App then fails loudly when the server starts, not as a confusing 502 on someone's first live request. Tokens are short-lived and cheap to re-request, so there's no local token-cache file to manage at all, unlike Gmail/Calendar/Excel-on-Drive.

3.4 Jira — the simplest auth of the five

```
class JiraClient(Protocol):
    def search_issues(self, jql: str, max_results: int) -> list[dict[str, Any]]: ...
    def get_issue(self, issue_key: str) -> dict[str, Any]: ...
    def create_issue(self, fields: dict[str, Any]) -> dict[str, Any]: ...
    def update_issue(self, issue_key: str, fields: dict[str, Any]) -> dict[str, Any]: ...

class JiraTool(BaseTool):
    name = "jira"
    capabilities = frozenset({Capability.READ, Capability.WRITE})
```

`search_issues` runs a JQL query; `create_issue`/`update_issue` are the writes. Auth here is the simplest of all five: an Atlassian API token sent as HTTP Basic auth (account email + token) on every request — no token exchange, no expiry to manage, nothing to cache. Two quirks `JiraRestClient` works around, both found by testing against a real Jira Cloud site rather than just reading the docs: Jira Cloud's newer `/search/jql` endpoint rejects an "unbounded" query with no restriction clause at all — `health_check` queries `"assignee = currentUser()"` rather than a bare `ORDER BY` — and it omits `key/fields` from results unless a `fields` param is passed explicitly, so `search_issues` always requests `fields: ["*all"]` to match `get_issue`.

3.5 All five, side by side

Tool	Capabilities	Read methods	Write/action methods
gmail	READ, ACTION	search_emails, read_message	send_email
google_calendar	READ, ACTION	search_events, read_event	create_event
excel_file	READ, WRITE	list_worksheets, read_range	write_range
salesforce	READ, WRITE	query_records, get_record	create_record, update_record
jira	READ, WRITE	search_issues, get_issue	create_issue, update_issue

The ACTION vs WRITE split lines up with a real distinction: Gmail/Calendar trigger a one-shot side effect (send, create-and-notify), while Excel/Salesforce/Jira create-or-overwrite a persistent record you could just as easily read back afterward. Nothing in the code branches on that distinction today — it's documentation of intent via `Capability` (§0.3, §2.1), not enforced behavior.

3.6 How five independent connectors become one `dict[str, BaseTool]`

`api.dependencies.get_tools()` is where the connector layer stops being five separate modules and becomes the one object the rest of the system depends on:

```

@lru_cache
def get_tools() -> dict[str, BaseTool]:
    tools: dict[str, BaseTool] = {}
    try:
        gmail_tool, calendar_tool = build_gmail_and_calendar_tools(state, settings)
    except RuntimeError:
        pass # no Google client id/secret configured -- omit both, together
    else:
        tools["gmail"] = gmail_tool
        tools["google_calendar"] = calendar_tool

    excel_tool = build_configured_excel_tool(state, settings)
    if excel_tool is not None:
        tools["excel_file"] = excel_tool
    # ...salesforce, jira follow the same if-not-None pattern
    return tools

```

Three things worth naming explicitly. First, **each connector is independently optional** — a deployment configures whichever credentials it has, and `tools_routes.py`'s `_tool` helper (FastAPI doc §2.4) 503s only the routes for a missing tool, rather than the whole API failing to start. Second, **Gmail and Calendar are wired as a pair**, not independently — `build_gmail_and_calendar_tools` (§2.7's factory-function shape, applied to two tools at once) requests both tools' OAuth scopes in a single consent, because requesting them separately against the same token file would have each overwrite the other's cached token (`google_auth.py`'s docstring calls this out directly — a real bug, not a hypothetical). Excel's Drive source shares the same Google OAuth client id/secret but deliberately writes to its *own* token-cache file for exactly the same reason. Third, **execute_node (Action Graph doc §2.3) never special-cases any of this** — `tools[proposed["tool"]]` plus `getattr(tool, proposed["method"])` dispatches to whichever of the five connectors and whichever of its methods a caller named, generically, because every one of them honors the same `BaseTool` shape. That genericity — not any per-tool logic — is the entire point of Part 1's "one common interface" argument (§1.1).

3.7 Auth, side by side

Tool	Auth mechanism	Token caching
gmail	Google OAuth2, user consent	.google_token.json, shared with google_calendar
google_calendar	Google OAuth2, user consent (same flow as gmail)	same file as gmail
excel_file (Drive)	Google OAuth2, user consent, full drive scope	own file, .google_drive_token.json
excel_file (local)	none	n/a
salesforce	OAuth2 Client Credentials (server-to-server)	none -- short-lived, re-requested each process start
jira	Atlassian API token, HTTP Basic	none -- sent on every request

The pattern across all five: only Gmail/Calendar/Excel-on-Drive need an interactive human consent step at all, and only because they act as a real Google end user. Salesforce and Jira are both server-to-server from the start — no human ever "logs in" as them, which is also why their factories (§2.7's shape) need no `token_path` argument at all.

Part 4 — Design patterns used here, named explicitly

Pattern	Where	What problem it actually solves here
Template Method (via ABC)	BaseTool -- name, capabilities, health_check abstract; _log, require_dry_run_guard concrete	Every connector gets shared audit-logging/dry-run machinery for free, and Python enforces that health_check is never forgotten.
Structural typing (Protocol)	GmailClient(Protocol)	Lets the real API client and a test fake both satisfy the interface without sharing a base class -- contrast with BaseTool's nominal ABC, §1.2.
Value Object	ActionResult, EmailSummary -- both @dataclass(frozen=True)	Immutable, equality-comparable result objects -- once built, can't be silently mutated by something downstream.
Decorator (Retry)	with_retry() wrapping read-only client methods	Transient network failures get retried transparently, without touching the calling code at all -- and is trivially omissible for writes.
Defense in depth	Placeholder-domain refusal + require_dry_run_guard + the Action Graph's interrupt()	Three independent layers, each catching a different failure mode -- a fabricated recipient, a missed audit log, and an unapproved write, respectively.
Fail-safe default	dry_run: bool = True; health_check must never raise	Calling a write method with no explicit dry_run argument is always safe; a broken connector reports unhealthy instead of crashing a status check.
Factory Function	build_gmail_tool(state, settings)	Same shape as create_app()/build_action_graph() -- take dependencies, run setup, return a built object; tests bypass it entirely.
Strategy	WorkbookSource: LocalWorkbookSource vs. GoogleDriveWorkbookSource (§3.2)	ExcelFileTool reads/writes bytes through one interface without caring which backing store it is -- the source is swapped at construction time, not branched on at call time.
Optional component wiring	get_tools() (§3.6) -- each connector's build_configured_*_tool returns None when unconfigured	A deployment missing one tool's credentials loses only that tool's routes (a per-route 503), never the whole API failing to start.

Part 5 — Interview-prep Q&A

Practice answering by pointing at the actual code, not reciting the definition.

Q: This codebase uses both ABC and Protocol for 'interfaces'. What's the actual difference, and how did they choose which to use where?

A: ABC (BaseTool) is nominal typing -- a class must explicitly inherit to count, and Python enforces every abstract method is implemented before the class can even be instantiated. Protocol (GmailClient) is structural typing -- anything with matching methods qualifies, no inheritance needed. They used ABC for BaseTool because they own every connector and want completeness enforced; Protocol for the HTTP client because a real client and a test fake shouldn't need a shared base class just to both qualify (§1.2).

Q: Where exactly does dry_run live, and how many places actually check it?

A: It's a parameter on every write/action method, defaulting to True. `require_dry_run_guard` logs regardless of its value; the if `dry_run`: branch inside the method itself is what actually decides whether the real client call happens. The Action Graph's `execute_node` is the only place in the whole codebase that ever passes `dry_run=False`, and only after an approved `interrupt()` (§1.3, and see the Action Graph doc).

Q: Why is the retry decorator only applied to read methods?

A: Because reads are idempotent -- retrying a fetch after a dropped connection just fetches the same data again, safely. A write isn't idempotent: if the connection drops after the server already processed it but before the success response arrives, a blind retry risks sending or creating something twice. Better for a transient write failure to surface as a failure a human looks at than to silently retry into a duplicate (§1.4, §2.6).

Q: Walk through what happens if a reasoning layer proposes sending an email to 'customer@example.com'.

A: `send_email`'s first check, `_is_reserved_placeholder_address`, catches it immediately -- `example.com` is an RFC 2606 reserved documentation domain. It logs an `action_failed` event and returns `ActionResult(executed=False, ...)` with reason `'reserved_placeholder_domain'` -- before `dry_run` is even considered, and regardless of whether a human already approved it, because a human reviewing the payload has no way to recognize a fabricated address as fake (§2.5).

Q: What does `frozen=True` buy you on `ActionResult`, beyond what a plain `dataclass` gives?

A: Immutability -- once constructed, none of its fields can be reassigned; attempting to changes it raises an error. Appropriate here because `ActionResult` is a finished result handed back up the call chain, never meant to be mutated by something downstream (same reasoning as `RunOutcome` in the Action Graph doc).

Q: Why does `GoogleApiClient` import `googleapiclient` inside `__init__` instead of at the top of the file?

A: So importing `gmail_tool.py` at all -- including for unit tests, which use a fake client -- never requires `googleapiclient` to be installed. Only code paths that actually construct a real `GoogleApiClient` need that dependency present (§2.7, same reasoning as the FastAPI doc's Postgres-driver example).

Q: If you were adding a sixth connector, what would you actually have to implement?

A: Inherit from `BaseTool`, set name and capabilities, implement `health_check` (required by `@abstractmethod`), route every public method through `self._log`, and call `require_dry_run_guard` at the top of every write/action method before checking `dry_run`. If the connector wraps a real HTTP client, define a minimal Protocol for exactly the methods needed, the same way `GmailClient` does, rather than depending on the third-party SDK's full client type.

Q: Why are Gmail and Calendar built together in `get_tools()`, instead of each independently like Excel/Salesforce/Jira?

A: Because they share one Google OAuth consent screen and one token-cache file. Requesting Gmail's scopes and Calendar's scopes in two separate `load_credentials()` calls against the same cache file would have the second call's consent silently overwrite the first tool's cached token -- `google_auth.py`'s docstring documents this as a real failure mode, not a hypothetical. `build_gmail_and_calendar_tools` requests every scope both tools need in one call, so one consent covers both, and if that fails (no Google client id/secret configured), both are omitted together rather than one succeeding and the other silently missing (§3.6).

Q: Why does Salesforce use a completely different auth flow from Jira, when both are read/write connectors over HTTP?

A: They're solving different trust problems. Salesforce's Client Credentials Flow authenticates the app itself as a specific 'Run As' service user via a Connected App -- appropriate for org-level integrations where an admin explicitly provisions and scopes that user's permissions. Jira's API token is simpler because Atlassian's model ties the token directly to one human account's existing permissions -- no separate app registration or service-user concept needed. Neither needs a browser consent step, unlike Gmail/Calendar/Excel-on-Drive, because both are server-to-server from the start (§3.7).

Q: How does Excel support both a local file and a Google Drive file without ExcelFileTool having two code paths?

A: WorkbookSource (§3.2) is a Protocol with exactly three methods -- describe, read_bytes, write_bytes. ExcelFileTool only ever calls those three methods; it has no idea whether the bytes came from Path.read_bytes() or a Drive API download. LocalWorkbookSource and GoogleDriveWorkbookSource are two interchangeable Strategy implementations chosen once, at construction time, by build_local_excel_tool/build_gdrive_excel_tool -- the Strategy pattern, concretely.

Vocabulary cheat-sheet

Term	Meaning here
ABC / @abstractmethod	A base class Python won't let you instantiate until every @abstractmethod is implemented by a subclass (§0.1).
Protocol	A structural-typing interface -- any class with matching methods qualifies, no inheritance required (§0.2).
Enum	A fixed, named set of values, safer against typos than plain string constants (§0.3).
Decorator factory	A function that, when called with arguments, returns a decorator -- e.g. with_retry(attempts=5) (§0.4).
@staticmethod	A method that doesn't receive self -- grouped in the class for organization, not because it needs an instance (§0.5).
Idempotent	An operation where doing it multiple times has the same effect as doing it once -- reads generally are, writes generally aren't (§1.4).
dry_run	A call that runs all the same logic as a real one but stops short of the actual side effect, reporting what would have happened (§1.3).
Defense in depth	Multiple independent checks, each catching a different failure mode, none redundant with the others (§2.5).
Adapter	A class translating one interface (a third-party SDK) into another (this repo's own minimal Protocol) -- GoogleApiClient is the example.

That's the whole layer: one shared contract (BaseTool), one narrow retry helper, five connectors each with their own auth mechanism and quirks (§3.1-3.7) but all identically shaped from the outside, and one dict (get_tools()) that wires them into a running server — each connector's actual write path defended by the same three independent layers, whether it's sending an email or updating a Salesforce record.